

Research Plan: Energy Benchmarking of KV Cache Compression Methods on GPU

Researcher: Koushik Bhupathiraju **Supervisor:** Prof. Taskin Kocak, Prof. Rongyu Lin **Timeline:** Spring 2026 Semester (January - May) **Deliverable:** Conference paper submission (target: ACM e-Energy 2026, HotCarbon 2026, or MLSys Workshop)

1. Research Problem

KV Cache compression has become essential for efficient LLM inference. As context lengths grow (32K, 128K, even 1M tokens), the KV Cache becomes a massive memory bottleneck. Researchers have developed numerous compression techniques:

- **Eviction methods:** H2O, StreamingLLM, SnapKV, NACL
- **Quantization methods:** KIVI (2-bit, 4-bit), KVQuant, PALU
- **Hybrid methods:** EvicPress, ThinKV

These methods report impressive results: **80% memory reduction, 2x throughput improvement, 54% KV cache reduction.**

However, a critical question remains unanswered:

Do KV Cache compression methods actually save energy, or do the compression/decompression overheads increase total power consumption?

This matters because: - Data centers pay for electricity, not memory - Compression requires additional compute (attention score calculation, quantization/dequantization) - “Faster inference” does not automatically mean “lower energy” - Different methods have fundamentally different compute-memory trade-offs

2. Research Gap (Validated from Literature Review)

Systematic Literature Search Results:

Paper	Year	What They Measured	Energy Gap
H2O (NeurIPS 2023)	2023	Throughput 29x, latency 1.9x, memory 80% reduction	No energy measurement
StreamingLLM	2023	Long sequence stability, fixed memory	No energy measurement
SnapKV	2024	Memory compression, accuracy, throughput	No energy measurement
KIVI (ICML 2024)	2024	2-bit quantization accuracy, memory	No energy measurement
PyramidInfer (ACL 2024)	2024	Throughput 2.2x, memory 54% reduction	No energy measurement
PALU (ICLR 2025)	2025	91% compression, perplexity	No energy measurement
EvicPress	2025	Joint compression + eviction	No energy measurement
TokenPowerBench (AAAI 2026)	2025	LLM inference joules/token	Does NOT test KV compression

Paper	Year	What They Measured	Energy Gap
From Prompts to Power	2025	155 models energy benchmark	Does NOT test KV compression
Kelle (MICRO 2025)	2025	4.5x energy efficiency	Custom eDRAM hardware only

Key Finding from “From Prompts to Power” (arXiv:2511.05597):

“The researchers conducted measurements using vLLM’s standard configuration **without exploring alternative KV cache optimization techniques** that might reduce energy demands.”

Key Finding from “A Systematic Characterization of LLM Inference” (arXiv:2512.01644):

The paper “does **not** test specific KV cache compression methods like H2O, StreamingLLM, SnapKV, or KIVI” and only recommends them as optimization guidelines without energy data.

Our Contribution:

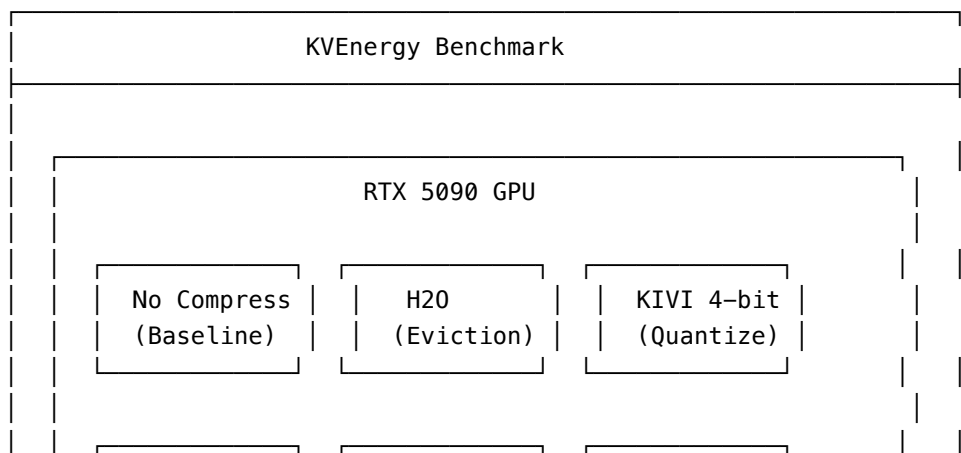
First systematic energy benchmark of KV Cache compression methods on standard GPUs, measuring actual joules/token and answering whether these widely-adopted techniques are truly energy-efficient.

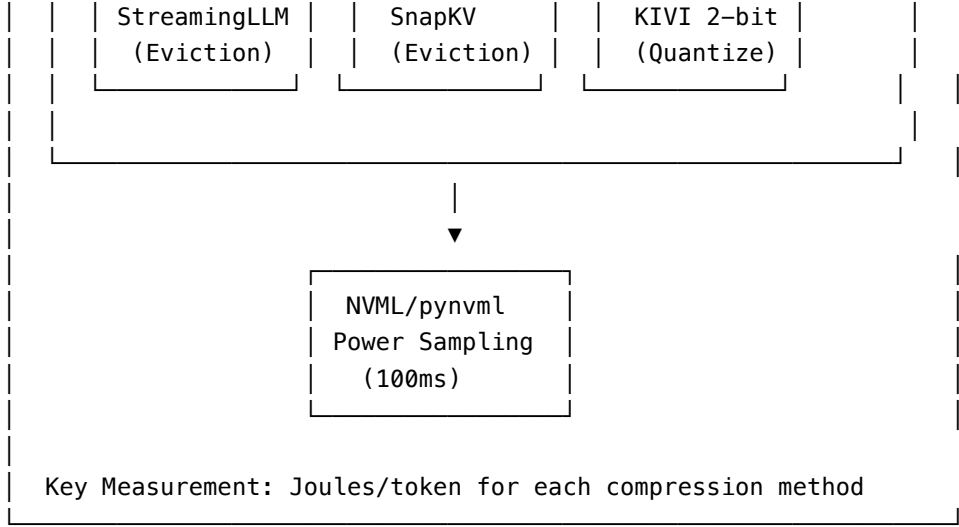
3. Research Questions

1. **RQ1:** What is the energy cost (joules/token) of different KV Cache compression methods (eviction vs. quantization vs. hybrid) compared to no compression?
2. **RQ2:** How does compression ratio correlate with energy efficiency? Does more aggressive compression (e.g., 2-bit vs. 4-bit) save or cost more energy?
3. **RQ3:** Which KV Cache compression method provides the best energy-efficiency (tokens/joule) for different context lengths and batch sizes?

4. Proposed Approach: KVEnergy Benchmark

4.1 Research Design Overview





4.2 KV Cache Methods to Benchmark

Category	Method	Compression Ratio	Compute Overhead
Baseline	No compression (FP16)	1x	None
Eviction	H2O (Heavy-Hitter Oracle)	5x	Attention score tracking
Eviction	StreamingLLM	Fixed size	Minimal (sink tokens)
Eviction	SnapKV	5-10x	Observation window voting
Quantization	KIVI 4-bit	4x	Quantize/dequantize
Quantization	KIVI 2-bit	8x	Quantize/dequantize
Hybrid	H2O + KIVI 4-bit	20x	Both overheads

4.3 Energy Measurement Methodology

Hardware Power Measurement: - NVML (**nvidia-smi**) via pynvml Python library - Sample rate: 100ms (GPU power reporting resolution) - Metrics: Total energy (Joules), Peak power (Watts), Average power (Watts)

Energy Attribution:

$E_{\text{total}} = \int P(t) dt$ (power integrated over inference time)
 $E_{\text{per_token}} = E_{\text{total}} / \text{num_tokens_generated}$
 $\text{Efficiency} = \text{tokens_generated} / E_{\text{total}}$ (tokens/joule)

Measurement Protocol: 1. Warm-up: 10 inference runs (discard) 2. Measurement: 50 inference runs per configuration 3. Idle power subtraction: Measure baseline GPU idle power 4. Report: Mean $\pm$ std for all metrics

5. Methodology

5.1 Hardware Setup

Component	Specification	Role
GPU	NVIDIA RTX 5090 (Lab)	All inference experiments
Power Monitoring	NVML via pynvml	GPU power measurement
Framework	vLLM / HuggingFace Transformers	Inference engine
KV Compression	KVCache-Factory library	Unified compression implementation

5.2 Model Configurations

Model	Parameters	Why
Llama 3.1 8B	8B	Popular, fits in GPU, well-supported
Qwen2.5 7B	7B	Alternative architecture
Mistral 7B v0.3	7B	Different attention pattern (sliding window)

5.3 Experimental Variables

Independent Variables: - KV compression method: 7 methods (including baseline) - Context length: 2K, 8K, 16K, 32K tokens - Batch size: 1, 4, 8, 16 - Output length: 50, 100, 200 tokens

Dependent Variables: - Energy per token (Joules/token) - Tokens per joule (efficiency) - Throughput (tokens/second) - Memory usage (GB) - Accuracy (perplexity on validation set)

5.4 Evaluation Tasks

Task	Dataset	Purpose
Long-context QA	NarrativeQA	Test long KV cache scenarios
Summarization	CNN/DailyMail	Typical inference workload
Multi-turn Dialogue	MT-Bench	Growing KV cache simulation
Perplexity	WikiText-2	Accuracy baseline

5.5 Baseline Comparisons

1. **No Compression:** Standard FP16 KV Cache (upper bound memory, baseline energy)
2. **vLLM Default:** Production default settings
3. **Per-Method Optimal:** Each compression method at its recommended settings

6. Research Timeline

Phase	Period	Milestone	Deliverable
Literature Review	Week 1-2	Deep dive into H2O, KIVI, TokenPowerBench papers	Annotated bibliography
Environment Setup	Week 3-4	KVCache-Factory + pynvml working	Validated measurement pipeline
Baseline Experiments	Week 5-6	No-compression energy baseline	Baseline energy data
Eviction Methods	Week 7-8	H2O, StreamingLLM, SnapKV energy	Eviction method data
Quantization Methods	Week 9-10	KIVI 4-bit, 2-bit energy	Quantization method data
Context Length Study	Week 11-12	Scaling behavior analysis	Complete experimental data
Analysis & Writing	Week 13-14	Statistical analysis, paper draft	Draft manuscript
Revision & Submission	Week 15-16	Incorporate feedback, submit	Camera-ready paper

7. Expected Contributions

1. **First energy benchmark** of KV Cache compression methods on standard GPUs (joules/token)
2. **Quantified trade-off analysis** between compression compute overhead and memory bandwidth savings
3. **Practical guidelines** for selecting energy-optimal KV compression for different workloads
4. **Open-source benchmark code** for reproducibility

8. Hypothesis & Expected Results

Primary Hypothesis: KV Cache compression methods that reduce memory bandwidth (quantization) will save energy, while methods with high compute overhead (complex eviction) may increase energy despite memory savings.

Expected Findings:

Method	Memory Savings	Expected Energy Impact	Rationale
StreamingLLM	Fixed	Save 10-20%	Minimal compute overhead
H2O	80%	Neutral or slight save	Attention tracking overhead
SnapKV	80%	Neutral	Observation window compute
KIVI 4-bit	75%	Save 20-30%	Reduced memory bandwidth
KIVI 2-bit	87.5%	Save or Cost	Aggressive quantization overhead

Key Insight to Validate: > Memory bandwidth is a major energy consumer in LLM inference. Methods that reduce bandwidth (quantization) should save energy, but the quantization/dequantization compute must not exceed the savings.

9. Risk Mitigation

Risk	Mitigation
Power measurement noise	Multiple runs, statistical analysis, idle power subtraction
Library incompatibility	Use KVCache-Factory which unifies all methods
No significant energy difference	Still valuable characterization; identify when differences emerge
Limited novelty	Extend to multiple GPUs (A100, 4090), different model families

10. Resources Required

- ☒ Lab GPU server (existing) - RTX 5090
- ☒ NVML/pynvml - Free (NVIDIA driver built-in)
- ☒ KVCache-Factory library - Open source
- ☒ vLLM / Transformers - Open source
- ☐ HuggingFace model downloads - Free (Llama access required)

Total budget: \$0 (all software-based, using existing hardware)

11. Success Criteria

Outcome	Minimum	Target	Stretch
Methods benchmarked	4	7	10+ including new 2025 methods
Context lengths tested	2	4	6+
Models tested	1	3	5+
Paper status	Workshop	Conference	Accept + artifact badge

12. Key References

KV Cache Compression Methods

1. **H2O (NeurIPS 2023)** - Heavy-Hitter Oracle for KV eviction
2. **StreamingLLM (2023)** - Attention sink + sliding window
3. **SnapKV (2024)** - Observation window voting
4. **KIVI (ICML 2024)** - Asymmetric 2-bit KV quantization
5. **PyramidInfer (ACL 2024)** - Layer-wise KV compression
6. **PALU (ICLR 2025)** - Low-rank KV compression

Energy Benchmarking

7. **TokenPowerBench (AAAI 2026)** - LLM inference energy benchmark (no KV compression)
8. **From Prompts to Power (2025)** - Large-scale energy study (no KV compression)
9. **Kelle (MICRO 2025)** - KV cache energy on custom eDRAM hardware

Resources

10. **KVCache-Factory** - Unified implementation: <https://github.com/Zefan-Cai/KVCache-Factory>
11. **Awesome-KV-Cache-Compression** - Paper collection: <https://github.com/October2001/Awesome-KV-Cache-Compression>

13. Why This Research Matters

This research addresses a **critical blind spot** in the KV Cache compression literature:

“KV Cache compression reduces memory, but does it reduce energy?”

All existing papers report memory reduction and throughput gains, but **none measure actual energy consumption**. For data centers where electricity is a major cost, and for sustainability concerns around AI’s carbon footprint, energy efficiency is crucial.

Our benchmark will answer: - Which compression method is most energy-efficient? - When does compression overhead exceed bandwidth savings? - How should practitioners choose compression methods for green AI?

14. Differentiation from Ranjot’s Research

Aspect	Ranjot	Koushik
Focus	Speculative Decoding energy	KV Cache Compression energy
Hardware	Edge + Cloud (Pi + GPU)	GPU server only
Variable	Speculation depth, network	Compression method, context length
Measurement	Joulescope (hardware meter)	NVML (software-based)
Cost	~\$150 (edge hardware)	\$0 (software only)

Both address energy efficiency gaps but from complementary angles.

15. Weekly Check-ins

- **Lab Meeting:** Tuesday 3:20 PM - Progress update (5 min)
- **1-on-1:** Book via Prof. Lin’s calendar for technical discussions
- **Async:** Post in lab Slack channel for quick questions

Research plan created: February 2026 Based on validated research gap: No existing joules/token measurements for KV Cache compression methods on standard GPUs Evidence: TokenPowerBench (2025), From Prompts to Power (2025) explicitly do not test KV compression; all KV compression papers only report memory/throughput